

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: "Информационная безопасность / Information security"

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Реализация БД в рамках СУБД»

Выполнил студент:

N3346 / ИББД_ТЗИ_N3 1.4

Суханкулиев Мухаммет / 

ФИО

Подпись

Проверил:

Салихов Максим Русланович / _____

ФИО

Подпись

Отметка о выполнении (один из вариантов:
отлично, хорошо, удовлетворительно, зачтено)

Дата

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Выбор систумы управления базами данных	4
2 Создание и наполнение базы данных	5
2.1 Создание базы данных	5
2.2 Код для создания таблиц	5
2.3 Код для внесения данных в таблицы	8
2.4 Модификация структуры таблицы	9
3 Индексирование таблиц	10
4 Установка взаимосвязей между таблицами	11
5 Создание представлений	12
5.1 Потребитель «Пользователи»	12
5.1.1 Представление 1. «История моих поездок»	12
5.1.2 Представление 2. «Доступные самокаты»	12
5.2 Потребитель «Технический специалист»	13
5.2.1 Представление 1. «Самокаты для зарядки»	13
5.2.2 Представление 2. «Журнал обслуживания самоката»	13
6 Тестирование БД	14
6.1 Подготовка окружения	14
6.2 Тесты	14
6.3 Запуск тестов и результаты	15
Заключение	16
Список использованных источников	17

ВВЕДЕНИЕ

Цель работы – получение навыков по работе с современными системами управления базами данных.

Для достижения поставленной цели необходимо решить следующие задачи:

- Выбрать систему управления базами данных (СУБД), которая будет использована в рамках лабораторной работы. Кратко обосновать свой выбор.
- Создать БД в выбранной СУБД на основе итоговой разработанной схемы отношений из ЛР 1. Заполнить созданную БД информацией, сгенерировать как минимум 7–8 кортежей с данными для каждой из основных таблиц. В отчете по лабораторной работе указать следующий SQL-код:
 - код для создания всех таблиц;
 - код для внесения данных в созданные таблицы;
 - код хотя бы одной SQL-команды для модифицирования структуры таблицы;
- Индексировать таблицы. Добавить индексы для атрибутов, по которым происходит объединение таблиц, а также атрибуты, по которым выполняется поиск/фильтрация данных.
- Установить взаимосвязи между таблицами.
- Дополнительно. 5 тестовых запросов к вашей БД
- Создать представления, составленные в пункте 5 лабораторной работы 1.

1 ВЫБОР СИСТЕМЫ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ

В качестве СУБД для реализации проекта была выбрана **PostgreSQL**. Этот выбор обоснован следующими причинами:

- **Надежность и соответствие стандартам** – это критически важно для систем, обрабатывающих финансовые транзакции и персональные данные.
- Она поддерживает **широкий спектр типов данных** (включая геометрические данные для хранения координат зон), сложные запросы и мощные механизмы индексации, что идеально подходит для задач нашей информационной системы (ИС).
- PostgreSQL предоставляет развитые **механизмы управления доступом**, включая ролевую модель доступа (RBAC), SSL-шифрование соединений и возможности для аудита.
- Будучи **бесплатным** и открытым решением, PostgreSQL позволяет избежать лицензионных ограничений и имеет активное сообщество, что упрощает поиск документации и решение возникающих проблем.

Для взаимодействия с СУБД будет использоваться графический интерфейс **pgAdmin 4**, который является стандартным инструментом для администрирования PostgreSQL.

2 СОЗДАНИЕ И НАПОЛНЕНИЕ БАЗЫ ДАННЫХ

2.1 Создание базы данных

Первым шагом создадим новую базу данных с названием `scooter_rental_db` для нашего проекта.

```
CREATE DATABASE scooter_rental_db  
  
WITH  
  
OWNER = postgres  
ENCODING = 'UTF8'  
CONNECTION LIMIT = -1  
IS_TEMPLATE = False;
```

2.2 Код для создания таблиц

На основе схемы отношений, разработанной в ЛР1, создадим таблицы.

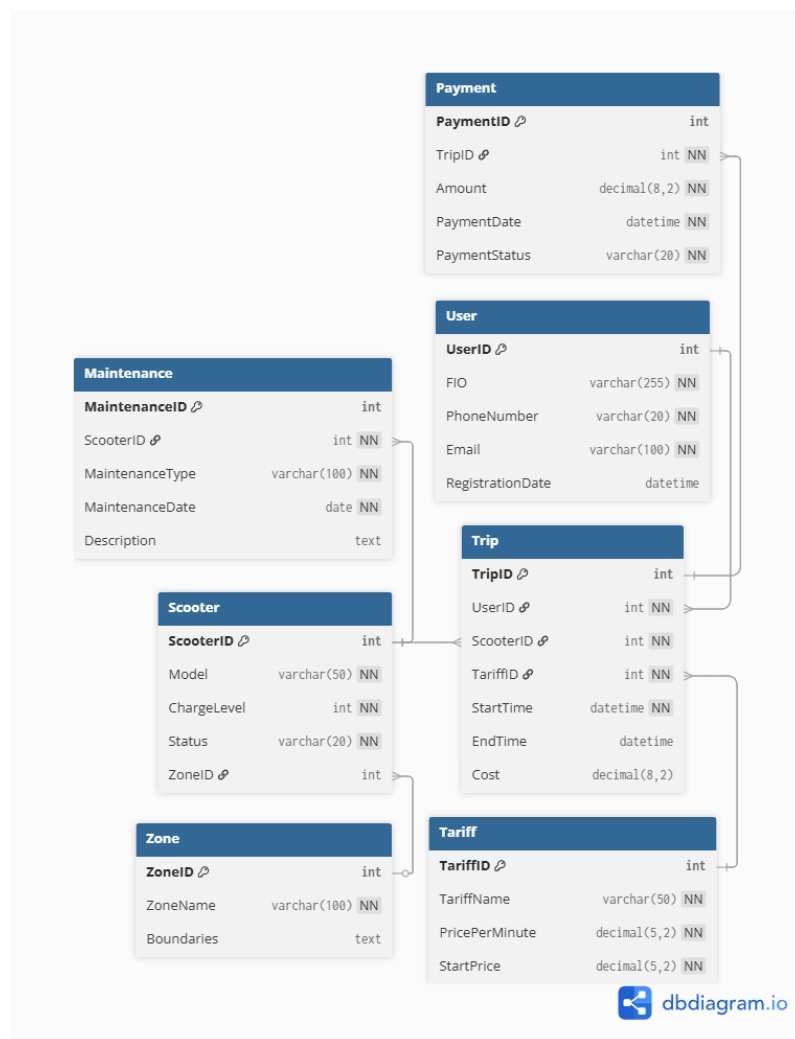


Рисунок 1 – ER-диаграмма

Таблица для разрешенных географических зон

```
CREATE TABLE IF NOT EXISTS public.Zone (  
    ZoneID serial PRIMARY KEY,  
    ZoneName varchar(100) NOT NULL,  
    Boundaries text -- Предполагается хранение в формате GeoJSON  
);
```

Таблица тарифов на аренду

```
CREATE TABLE IF NOT EXISTS public.Tariff (  
    TariffID serial PRIMARY KEY,  
    TariffName varchar(50) NOT NULL,  
    PricePerMinute decimal(5, 2) NOT NULL,  
    StartPrice decimal(5, 2) NOT NULL DEFAULT 0.00  
);
```

Таблица пользователей (клиентов) сервиса

```
CREATE TABLE IF NOT EXISTS public."User" (  
    UserID serial PRIMARY KEY,  
    FIO varchar(255) NOT NULL,  
    PhoneNumber varchar(20) UNIQUE NOT NULL,  
    Email varchar(100) UNIQUE NOT NULL,  
    RegistrationDate timestamp NOT NULL DEFAULT now()  
);
```

Таблица для хранения информации об электросамокатах

```
CREATE TABLE IF NOT EXISTS public.Scooter (  
    ScooterID serial PRIMARY KEY,  
    Model varchar(50) NOT NULL,  
    ChargeLevel int NOT NULL,  
    Status varchar(20) NOT NULL DEFAULT 'available',  
    ZoneID int,  
    CONSTRAINT fk_zone  
        FOREIGN KEY (ZoneID)  
        REFERENCES Zone (ZoneID)  
);
```

Таблица для учета работ по техническому обслуживанию

```
CREATE TABLE IF NOT EXISTS public.Maintenance (  
    MaintenanceID serial PRIMARY KEY,  
    ScooterID int NOT NULL,
```

```

MaintenanceType varchar(100) NOT NULL,
MaintenanceDate date NOT NULL,
Description text,
CONSTRAINT fk_scooter
    FOREIGN KEY(ScooterID)
    REFERENCES Scooter(ScooterID)
);

```

Таблица поездок, фиксирующая каждую аренду

```

CREATE TABLE IF NOT EXISTS public.Trip (
    TripID serial PRIMARY KEY,
    UserID int NOT NULL,
    ScooterID int NOT NULL,
    TariffID int NOT NULL,
    StartTime timestamp NOT NULL,
    EndTime timestamp,
    Cost decimal(8, 2),
    CONSTRAINT fk_user
        FOREIGN KEY(UserID)
        REFERENCES "User"(UserID),
    CONSTRAINT fk_scooter
        FOREIGN KEY(ScooterID)
        REFERENCES Scooter(ScooterID),
    CONSTRAINT fk_tariff
        FOREIGN KEY(TariffID)
        REFERENCES Tariff(TariffID)
);

```

Таблица платежей по поездкам

```

CREATE TABLE IF NOT EXISTS public.Payment (
    PaymentID serial PRIMARY KEY,
    TripID int NOT NULL,
    Amount decimal(8, 2) NOT NULL,
    PaymentDate timestamp NOT NULL DEFAULT now(),
    PaymentStatus varchar(20) NOT NULL DEFAULT 'completed',
    CONSTRAINT fk_trip
        FOREIGN KEY(TripID)
        REFERENCES Trip(TripID)
);

```

2.3 Код для внесения данных в таблицы

Заполним созданные таблицы тестовыми данными.

Заполнение таблицы Zone

```
-- С упрощенными полигонами
INSERT INTO public.Zone (ZoneName, Boundaries) VALUES
('Центральный район', '{"type":"Polygon", "coordinates": [[[30.35, 59.93],
    [30.35, 59.94], [30.38, 59.94], [30.38, 59.93], [30.35, 59.93]]]}'),
('Адмиралтейский район', '{"type":"Polygon", "coordinates": [[[30.28, 59.91],
    [30.28, 59.92], [30.31, 59.92], [30.31, 59.91], [30.28, 59.91]]]}'),
('Петроградский район', '{"type":"Polygon", "coordinates": [[[30.28, 59.96],
    [30.28, 59.97], [30.30, 59.97], [30.30, 59.96], [30.28, 59.96]]]}');
```

Заполнение таблицы Tariff

```
INSERT INTO public.Tariff (TariffName, PricePerMinute, StartPrice) VALUES
('Стандарт', 5.00, 50.00),
('Эко', 4.50, 30.00),
('Премиум', 7.50, 70.00);
```

Заполнение таблицы "User"

```
INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES
('Иванов Иван Иванович', '+79211234567', 'ivanov@email.com'),
('Петрова Мария Сергеевна', '+79119876543', 'petrova@email.com'),
('Сидоров Алексей Петрович', '+79055554433', 'sidorov@email.com'),
('Смирнова Ольга Викторовна', '+79998887766', 'smirnova@email.com'),
('Кузнецов Дмитрий Андреевич', '+79213332211', 'kuznetsov@email.com'),
('Васильева Анна Михайловна', '+79114445566', 'vasilieva@email.com'),
('Михайлов Сергей Александрович', '+79067778899', 'mikhailov@email.com'),
('Новикова Екатерина Дмитриевна', '+79211110022', 'novikova@email.com');
```

Заполнение таблицы Scooter

```
INSERT INTO public.Scooter (Model, ChargeLevel, Status, ZoneID) VALUES
('Ninebot G30', 85, 'available', 1),
('Kugoo M4', 100, 'available', 1),
('Xiaomi M365', 45, 'available', 2),
('Ninebot G30', 25, 'low_battery', 3),
('Xiaomi M365', 95, 'in_use', 1),
('Kugoo M4', 70, 'available', 2),
('Ninebot G30', 15, 'maintenance', 3),
('Xiaomi M365', 80, 'available', 1);
```


Заполнение таблицы Maintenance

```
INSERT INTO public.Maintenance (ScooterID, MaintenanceType, MaintenanceDate,
    Description) VALUES
(7, 'Замена тормозного диска', '2025-09-27', 'Плановая замена'),
(4, 'Зарядка аккумулятора', '2025-09-28', 'Низкий уровень заряда'),
(1, 'Проверка давления в шинах', '2025-09-25', 'Плановое ТО');
```

Заполнение таблицы Trip

```
INSERT INTO public.Trip (UserID, ScooterID, TariffID, StartTime, EndTime, Cost)
    VALUES
(1, 5, 1, '2025-09-28 10:00:00', '2025-09-28 10:15:00', 125.00),
(2, 3, 2, '2025-09-28 11:30:00', '2025-09-28 11:50:00', 120.00),
(3, 1, 1, '2025-09-27 15:00:00', '2025-09-27 15:30:00', 200.00),
(1, 2, 3, '2025-09-27 18:20:00', '2025-09-27 18:30:00', 145.00),
(4, 6, 1, '2025-09-28 09:00:00', '2025-09-28 09:12:00', 110.00),
(5, 8, 2, '2025-09-28 12:00:00', '2025-09-28 12:25:00', 142.50),
(2, 1, 1, '2025-09-28 13:00:00', '2025-09-28 13:05:00', 75.00),
(6, 2, 1, '2025-09-28 14:10:00', '2025-09-28 14:40:00', 200.00);
```

Заполнение таблицы Payment

```
INSERT INTO public.Payment (TripID, Amount, PaymentStatus) VALUES
(1, 125.00, 'completed'),
(2, 120.00, 'completed'),
(3, 200.00, 'completed'),
(4, 145.00, 'completed'),
(5, 110.00, 'completed'),
(6, 142.50, 'completed'),
(7, 75.00, 'completed'),
(8, 200.00, 'completed');
```

2.4 Модификация структуры таблицы

Добавим в таблицу "User" новое поле LastLogin для отслеживания времени последнего входа пользователя в систему.

```
ALTER TABLE public."User"
ADD COLUMN LastLogin timestamp;
```

3 ИНДЕКСИРОВАНИЕ ТАБЛИЦ

Для ускорения выполнения запросов, особенно операций JOIN и WHERE, добавим индексы на внешние ключи и часто используемые для поиска поля.

Индексы для таблицы Scooter

```
CREATE INDEX idx_scooter_zoneid ON public.Scooter (ZoneID);  
CREATE INDEX idx_scooter_status ON public.Scooter (Status);
```

Индексы для таблицы Maintenance

```
CREATE INDEX idx_maintenance_scooterid ON public.Maintenance (ScooterID);
```

Индексы для таблицы Trip

```
CREATE INDEX idx_trip_userid ON public.Trip (UserID);  
CREATE INDEX idx_trip_scooterid ON public.Trip (ScooterID);  
CREATE INDEX idx_trip_tariffid ON public.Trip (TariffID);
```

Индексы для таблицы Payment

```
CREATE INDEX idx_payment_tripid ON public.Payment (TripID);
```

4 УСТАНОВКА ВЗАИМОСВЯЗЕЙ МЕЖДУ ТАБЛИЦАМИ

Взаимосвязи между таблицами в реляционной модели данных реализованы с помощью механизма внешних ключей. Этот механизм обеспечивает ссылочную целостность данных, гарантируя, что связанные данные остаются согласованными. Например, невозможно создать запись о поездке для пользователя, которого не существует в системе.

В разработанной базе данных `scooter_rental_db` были установлены следующие ключевые взаимосвязи при создании таблиц:

- **Scooter → Zone (1:N):** Каждый самокат в определенный момент времени находится в одной географической зоне. Связь установлена через внешний ключ `Scooter.ZoneID`, ссылающийся на `Zone.ZoneID`.
- **Maintenance → Scooter (1:N):** Для одного самоката может быть множество записей о его техническом обслуживании. Связь установлена через внешний ключ `Maintenance.ScooterID`, ссылающийся на `Scooter.ScooterID`.
- **Trip → User (1:N):** Один пользователь может совершить множество поездок. Связь установлена через внешний ключ `Trip.UserID`, ссылающийся на `"User".UserID`.
- **Trip → Scooter (1:N):** Один самокат может быть использован во множестве поездок. Связь установлена через внешний ключ `Trip.ScooterID`, ссылающийся на `Scooter.ScooterID`.
- **Trip → Tariff (1:N):** Множество поездок могут тарифицироваться по одному тарифу. Связь установлена через внешний ключ `Trip.TariffID`, ссылающийся на `Tariff.TariffID`.
- **Payment → Trip (1:N):** Одна поездка может иметь один или несколько связанных платежей. Связь установлена через внешний ключ `Payment.TripID`, ссылающийся на `Trip.TripID`.

Все эти ограничения были определены с помощью конструкции `CONSTRAINT FOREIGN KEY` в SQL-скриптах создания таблиц, представленных в разделе 2.2.

5 СОЗДАНИЕ ПРЕДСТАВЛЕНИЙ

Создадим представления, спроектированные в ЛР1, для упрощения доступа к данным для разных классов потребителей.

5.1 Потребитель «Пользователи»

5.1.1 Представление 1. «История моих поездок»

```
CREATE VIEW user_trip_history AS
SELECT
    t.TripID,
    t.UserID,
    t.StartTime AS "Дата поездки",
    s.Model AS "Модель самоката",
    (t.EndTime - t.StartTime) AS "Длительность",
    t.Cost AS "Стоимость",
    p.PaymentStatus AS "Статус платежа"
FROM Trip t
JOIN Scooter s ON t.ScooterID = s.ScooterID
JOIN Payment p ON t.TripID = p.TripID
```

5.1.2 Представление 2. «Доступные самокаты»

```
CREATE VIEW available_scooters AS
SELECT
    s.ScooterID,
    z.ZoneName AS "Местоположение",
    s.ChargeLevel AS "Уровень заряда",
    tr.TariffName AS "Тариф",
    tr.PricePerMinute AS "Стоимость минуты"
FROM Scooter s
JOIN Zone z ON s.ZoneID = z.ZoneID
-- Присоединяем тариф через таблицу поездок
-- (представим, что все самокаты в зоне работают по первому тарифу)
JOIN Tariff tr ON tr.TariffID = 1
WHERE s.Status = 'available';
```

5.2 Потребитель «Технический специалист»

5.2.1 Представление 1. «Самокаты для зарядки»

```
CREATE VIEW scooters_for_charging AS
SELECT
    s.ScooterID,
    s.Model,
    s.ChargeLevel AS "Текущий уровень заряда",
    z.ZoneName AS "Название зоны"
FROM Scooter s
JOIN Zone z ON s.ZoneID = z.ZoneID
WHERE s.Status = 'available' AND s.ChargeLevel < 30
ORDER BY s.ChargeLevel ASC;
```

5.2.2 Представление 2. «Журнал обслуживания самоката»

```
CREATE VIEW maintenance_log AS
SELECT
    s.ScooterID,
    s.Model,
    m.MaintenanceDate AS "Дата обслуживания",
    m.MaintenanceType AS "Тип работ",
    m.Description AS "Описание работ"
FROM Maintenance m
JOIN Scooter s ON m.ScooterID = s.ScooterID
ORDER BY s.ScooterID, m.MaintenanceDate DESC;
```

6 ТЕСТИРОВАНИЕ БД

Для проверки корректности созданной схемы, целостности данных и работоспособности бизнес-логики (реализованной в представлениях) были написаны автоматизированные тесты с использованием фреймворка `pytest` и библиотеки `psycopg2` для подключения к PostgreSQL.

6.1 Подготовка окружения

Для запуска тестов необходимо установить следующие Python-библиотеки:

```
pip install pytest
pip install psycopg2-binary
```

6.2 Тесты

Тесты размещены в файле `test_db.py` (Приложение А). Используется механизм фикстур `pytest` для управления подключением к базе данных.

Запрос: `SELECT COUNT(*) FROM public."User";`

Ожидаемый результат: Запрос возвращает число, равное или большее 8, что подтверждает корректное выполнение скрипта для заполнения таблицы исходными данными.

Запрос: `SELECT "Текущий уровень заряда" FROM public.scooters_for_charging;`

Ожидаемый результат: Все числовые значения, возвращенные этим запросом, должны быть строго меньше 30. Это доказывает, что логический фильтр (`WHERE ChargeLevel < 30`) в представлении работает корректно.

Действие: Попытка выполнить SQL-запрос `INSERT INTO public.Trip (UserID, ScooterID, TariffID, StartTime) VALUES (999999, 1, 1, NOW());`, где `UserID = 999999` заведомо не существует в таблице `"User"`.

Ожидаемый результат: СУБД должна прервать выполнение запроса и сгенерировать ошибку нарушения целостности внешнего ключа (`ForeignKeyViolation`). Запись не должна быть добавлена в таблицу.

Действие: Попытка выполнить SQL-запрос `INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES ('Тестовый Пользователь', '+700000000000', 'ivanov@email.com');`, где `Email = 'ivanov@email.com'` уже существует в таблице.

Ожидаемый результат: СУБД должна прервать выполнение запроса и сгенерировать ошибку нарушения уникальности (UniqueViolation). Новая запись не должна быть добавлена в таблицу.

Действие: Выполняется INSERT в таблицу Scooter, намеренно пропуская столбец Status, для которого в схеме определено значение DEFAULT 'available'. Сразу после этого выполняется SELECT Status для только что вставленной записи.

Ожидаемый результат: Запрос на вставку выполняется успешно. Последующий SELECT возвращает значение 'available', что подтверждает корректную работу механизма DEFAULT в СУБД.

6.3 Запуск тестов и результаты

Для запуска тестов достаточно выполнить в терминале, находясь в каталоге с файлом test_db.py, команду:

```
pytest -v
```

```
C:\Users\muham\Desktop\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\np2>pytest -v
C:\Users\muham\AppData\Local\Programs\Python\Python313\Lib\site-packages\pytest_asyncio\plugin.py:252: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))

===== test session starts =====
platform win32 -- Python 3.13.2, pytest-8.3.5, pluggy-1.5.0 -- C:\Users\muham\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\muham\Desktop\itmo\_Studying(5th_sem)\DatabaseSecurity(2nd)\np2
plugins: anyio-4.8.0, locust-2.40.1, asyncio-1.2.0, cov-6.2.1, mock-3.15.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

test_db.py::test_connection_is_successful PASSED [ 16%]
test_db.py::test_users_table_is_populated PASSED [ 33%]
test_db.py::test_scooters_for_charging_view_logic PASSED [ 50%]
test_db.py::test_foreign_key_constraint PASSED [ 66%]
test_db.py::test_unique_email_constraint PASSED [ 83%]
test_db.py::test_scooter_status_default_value PASSED [100%]

===== 6 passed in 0.08s =====
```

Рисунок 2 – Результат тестирования

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы 2 была успешно создана и наполнена данными база данных `scooter_rental_db` в СУБД PostgreSQL. Были реализованы все таблицы в соответствии с инфологической моделью из ЛР1, установлены связи между ними, добавлены индексы для оптимизации запросов и созданы представления для различных категорий пользователей. Получены практические навыки по созданию, модификации и администрированию баз данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с. — URL: https://drive.google.com/file/d/1TjYbunEjxsbovBiHeYOzBuZrOFonlIRk/view?usp=drive_link
2. Базы данных: Учебник для высших учебных заведений / Под ред. проф. А. Д. Хомоненко. — 6-е изд., доп. - СПб.: КОРОНА-Век, 2009. — 736 с. — URL: https://drive.google.com/file/d/1zIOuO6vdQvb_aVUHGAiucK5MPciUswGf/view?usp=drive_link
3. Psycopg – PostgreSQL database adapter for Python : официальная документация [Электронный ресурс] / The Psycopg Team. — URL: <https://www.psycopg.org/docs/> (дата обращения: 28.09.2025)

ПРИЛОЖЕНИЕ А

Листинг А.1 – test_db.py

```
import pytest
import psycopg2

# --- Параметры подключения к БД ---
DB_PARAMS = {
    "host": "localhost",
    "port": 5252,
    "user": "postgres",
    "password": "пароль",
    "dbname": "scooter_rental_db"
}

@pytest.fixture(scope="module")
def db_connection():
    """Фикстура для создания и закрытия соединения с БД."""
    try:
        conn = psycopg2.connect(**DB_PARAMS)
        yield conn
        conn.close()
    except psycopg2.OperationalError as e:
        pytest.fail(f"Не удалось подключиться к базе данных: {e}")

def test_connection_is_successful(db_connection):
    """Проверяет, что соединение с БД успешно установлено."""
    assert db_connection is not None and not db_connection.closed

def test_users_table_is_populated(db_connection):
    """Проверяет, что таблица "User" содержит данные (не пустая)."""
    with db_connection.cursor() as cur:
        cur.execute('SELECT COUNT(*) FROM public."User"')
        user_count = cur.fetchone()[0]
        assert user_count >= 8, "В таблице User должно быть не менее 8 записей"

def test_scooters_for_charging_view_logic(db_connection):
    """Проверяет логику представления 'scooters_for_charging': все самокаты в нем должны иметь заряд < 30%. """
    with db_connection.cursor() as cur:
        cur.execute('SELECT "Текущий уровень заряда" FROM public.scooters_for_charging')
        low_charge_levels = cur.fetchall()
        for level in low_charge_levels:
            assert level[0] < 30, f"Найден самокат с уровнем заряда {level[0]}, что не соответствует логике представления"

def test_foreign_key_constraint(db_connection):
    """Проверяет работу ограничения внешнего ключа, пытаюсь вставить поездку с несуществующим UserID. """
    with pytest.raises(psycopg2.errors.ForeignKeyViolation):
        with db_connection.cursor() as cur:
            invalid_user_id = 999999
            cur.execute(
                "INSERT INTO public.Trip (UserID, ScooterID, TariffID, StartTime) VALUES (%s, 1, 1, NOW())",
                (invalid_user_id,)
            )
```

```

        db_connection.rollback()

def test_unique_email_constraint(db_connection):
    """Проверяет работу ограничения UNIQUE для Email в таблице "User",
    пытаюсь вставить дубликат."""
    with pytest.raises(psycopg2.errors.UniqueViolation):
        with db_connection.cursor() as cur:
            existing_email = 'ivanov@email.com'
            cur.execute(
                'INSERT INTO public."User" (FIO, PhoneNumber, Email) VALUES
                (%s, %s, %s)',
                ('Тестовый Пользователь', '+700000000000', existing_email)
            )
        db_connection.rollback()

def test_scooter_status_default_value(db_connection):
    """Проверяет, что для нового самоката по умолчанию устанавливается статус
    'available'."""
    with db_connection.cursor() as cur:
        cur.execute(
            "INSERT INTO public.Scooter (Model, ChargeLevel, ZoneID) VALUES
            (%s, %s, %s) RETURNING ScooterID",
            ('Test Model', 100, 1)
        )
        new_scooter_id = cur.fetchone()[0]

        cur.execute("SELECT Status FROM public.Scooter WHERE ScooterID = %s",
            (new_scooter_id,))
        status = cur.fetchone()[0]

        assert status == 'available'

    db_connection.rollback()

```